Letting AI Build an Engineering Tool: What to Expect

Pablo Magariños – *18 September 2026*

Keywords: AI Engineering, Aircraft Design

Summary. I built a first version of IDLEDrones, an optimiser for fixed-wing competition drones, with Codex and two main prompts: one to document the project and one to implement it. The process took almost three hours, but my active work was under 30 minutes. The result is a conceptual engineering tool that works and proposes designs that make sense, although it should still be taken with scepticism: the physics needs more validation and the interface needs rethinking. It is not public for now, because we are still working on it.

What I am doing

First of all, this is not **vibe coding** in the sense of asking for something in two lines and accepting whatever comes out. The AI wrote practically all of the code, but the weight of my work was in defining the problem well, answering its questions and reviewing the result with an engineer's judgement.

I wanted to build an aircraft optimisation tool for university fixed-wing drone competitions, where each team designs and builds its own aircraft to complete a flight mission: carry as much payload as possible, fly a circuit as fast as possible, or both, within a set of design rules. I called it *IDLEDrones*, for *I Don't Like Engineering: Drones*. These competitions have objective scoring systems, at least for the flight part; the **technical reports** and other sections are more subjective. So everything that is objective can be optimised, if you simplify it enough :)

This post is not a tutorial on building an application from scratch, nor a universal recipe: I have not done any master's degree in AI programming. I want to share what you can expect when you give an AI the ability to build an application with barely any human control, through the story of how this first version came out of two main prompts. Of course I cannot include all of my reasoning here, but I think I have captured the essentials, as well as what results a simple approach can get you, without an endless collection of extra tools.

Tools

I used Codex exclusively, specifically Codex CLI, that is, Codex from the terminal, although there is also a desktop app. I do it this way because when I started there was no app for Ubuntu and I got used to it. It also lets me open several Codex terminals in parallel and watch them all progress at once: sometimes two on the same project, or a few more if I am working on two projects at the same time.

I work on Ubuntu because Linux saves me some compatibility problems with scientific Python libraries. This particular version was built and verified on Linux; it can probably be adapted

to Windows or macOS, but I have not checked. To manage Python locally I recommend uv: anyone can download the project and run `uv sync` to get the right Python version and the pinned dependencies. And however much AI you use to program, version control is still essential. Being honest, in this project everything ended up in a single initial **commit**; it is from now on, iterating on this base, that Git becomes really useful.

The part that may surprise people with more experience: for this project I did not create any special `AGENTS.md`, any **skill** or any MCP. I used Codex practically **vanilla**, as it comes. My reason is simple: OpenAI does a much better job than I do at optimising these tools for this kind of task, and too many instructions can also degrade how they work. Don't tell it upfront how to do absolutely everything. Give it some freedom and restrict whatever you see failing.

In other projects I do usually create an `AGENTS.md` or a `CLAUDE.md` with simple instructions when I need them: use `uv` to manage Python, keep documentation in `docs/`, write code in English even though I write in Spanish, or question decisions that could be improved. I try not to overcomplicate it and only add rules I have seen work. Don't do it before you know you need it. That is my personal advice.

Prompt 1: I asked GPT-6 Astra for all the project documentation

At the time of writing (September 2026), I consider GPT-6 Astra to be state of the art, alongside Fable 5.1. I used *medium* effort: it is a very powerful model, and that level is usually enough for documentation without burning the whole weekly limit in one go. I wanted the model to detail the project and, above all, to raise every question it needed to. A more capable model tends to be better at spotting flaws, weak reasoning and alternative implementations. Afterwards, for the implementation, you can drop to a cheaper model.

When I build this kind of tool, I usually ask for very similar things:

1. I explain the overall goal. That way, if I go off on a tangent later, the model does not lose sight of what I am trying to achieve.
2. I ask for a Python backend and a web frontend. It saves me clunky interfaces, and FastAPI serves as the entry point to the real software. Python has an absurd number of useful tools, like NumPy, JAX or PyTorch, and it is also a language that is very present in these models' training data.
3. I explain how I think the problem could be solved, broadly. The more specific you are, and the more freedom you give it to challenge you, the more likely you both understand the same goal. Conveying a concrete vision in writing is hard: try to do it well, but let the model propose alternatives.

First I wrote my ideas in a file, `idea_inicial.md`. Both prompts are translated from Spanish:

Prompt: idea_inicial.md

> I want to build an optimiser of fixed-wing drones for drone competitions. The goal of the competition is to fly a circuit meeting several objectives. The goal of the optimisation is, therefore, to get the optimal drone for it. For this version, the optimisation circuit will be a 100-metre straight, a tight 20-metre-radius turn, and the same again, 10 times. So the optimisation has to give the optimal drone for

> that.
>
> For now, I was thinking of doing the optimisation with a genetic algorithm, iterating over drone designs. For the constraints you have to give me, on the web, a **drag and drop** option. Some examples: maximum wingspan, maximum drone length, maximum motor power (for this test we assume one motor), etc. Add more if you think of a key constraint.
>
> Then, under design, you have to give me options: rib material (balsa wood, aluminium, carbon, etc.), skin material (carbon fibre, vinyl, etc.), installed power (remember that if I install X power, I can't then use maximum power as a constraint; maybe I can also fix the motor weight here) and more things if you think of them. It is the space of possibilities that has to be chosen and that aren't trivial, mainly materials and motors.
>
> You have to model multiple things: battery weight as a function of its capacity and discharge rate, wing aerodynamics with fast but reliable numerical methods, structural capacity with fast methods based on wing loading, making the wing spar or spars carry it, the mass of the simplified internal structure (ribs and spars), the aerodynamics of ailerons and flaps as a function of their dimensions, and the drone's stability as a function of its dimensions, distances, CG, etc. Also remember that the goal may be to maximise payload, so take it into account during the optimisation.
>
> The idea is to optimise as much as possible and get the optimal drone for the proposed mission. The circuit is what it is, and the optimisation can go for payload, speed, energy efficiency or circuit time. So let me weight those things.
>
> I want the backend to run in Python as efficiently as possible to speed up computation, and to control everything from a website launched, probably, with FastAPI.

Then, in that folder, where only that file existed, I told the model this:

Prompt: documentation request

> I need to build a professional optimiser/simulator for my competition drone team. The v1 is in `idea_inicial`. Read it, analyse it in detail and write in `docs/` everything needed to build it, since you will delegate the construction to another AI. You have to detail it so that it works well, does what I ask, and catches things I may not have described: missing information, approaches that fail or aspects that might interest me. You must ask me about all of that and I will answer. You can ask me as you plan or before.

As you can see, I simply think out loud. I usually write as fast as possible, because the brain thinks faster than we write. I don't worry much about wording the prompt well: I try not to spend mental space on perfect writing.

The AI created `docs/` and got to work. On three or four occasions it came back with batches of questions: orders of magnitude for the dimensions, whether I wanted to model take-off, what the formula to maximise should look like, etc. The key here is to stay alert and think

long term: what problems will appear once the software is finished, whether it will actually be useful, or whether the scope is so ambitious that it will become unbearable. The answers to these questions have a big influence on whether the result is good or bad.

Thanks to those questions I came up with things I had not included in the initial prompt. My favourite was adding a Monte Carlo analysis of the solutions. As you can imagine, luck plays a big part in a competition. The v1 varies the wind and its direction; in the future I would also like to add the performance of rival teams. Many competitions have scores that depend on the best result in a category, for example the top speed. That can shift the real optimum and opens the door to game theory: the optimal drone may depend on whether everyone else maximises one score or several. That is material for another post, but game theory and modelling rivals are very interesting topics.

There is something I did not stress enough in the initial prompt and should have asked for from the start: explicit tests to validate the physics. The plan and the implementation ended up including them, but in a simulation tool it pays to demand from the very beginning unit tests for each model, comparisons against real data, and system tests that check the consistency between sub-simulations. It can also help to compare independent formulations or solvers and to study the error margins expected in the literature, although agreement between two solvers is no substitute for experimental validation.

Prompt 2: Putting GPT-5.6 Sol to implement it

Once the documentation was finished, I asked GPT-6 Astra to write a prompt for a new session and closed the original one. The idea was for the new agent to know how to execute the plan without getting lost in all the documentation. I don't include that prompt because the AI wrote it itself, and it saves space.

I ran GPT-5.6 Sol at *xhigh* effort so it would be more thorough with the tests and the search for information. Maybe *high* would have been enough, but I wanted something more powerful. I am in favour of spending a lot of effort when nothing exists yet, because afterwards it is harder to fix what has already been built.

As a reference for a midpoint: 40 minutes into the run it was inspecting PNG files generated by rendering the PDF reports. That is one of its advantages: being multimodal, it can visually check what it produces. That is why it is so interesting that these models can read images.

An hour and a half in (yes, that is quite long, partly because of *xhigh*), I had seen it delete code and rewrite it several times. That showed it was iterating, testing and building the project little by little, rather than just writing a single version and calling it done.

I should clarify something about usage limits. At the time of writing (September 2026), I use Codex on the Plus plan and, in my case, I don't have the five-hour limit, only the weekly one. That lets me run extremely long tasks; with Claude, on the other hand, I can't do it the same way. I know other people on Codex do have the five-hour limit, and that is fine: a project like this is still possible with shorter limits, it just takes longer. You can resume the task when your usage comes back, or keep a task file and update it as you go. Also, the longer a conversation gets, the more context it drags along and the higher the consumption can be. Keeping tasks small and well scoped can save usage by working with shorter contexts.

The whole process took almost three hours and used roughly 40 % of my weekly Codex usage. My active working time was under 30 minutes: mostly writing the prompts, answering questions, reviewing progress and checking the result. The rest was the model running autonomously.

Result (in short)

The result is a local Linux application, with a Python backend and web control, able to evaluate and optimise conventional aircraft, flying wings and canards. It does not just vary a geometry: it jointly closes aerodynamics, structure, mass and CG, stability, control surfaces, payload housing, motor, propeller, battery and mission. If a design cannot take off, trim, withstand the loads, deliver the power or complete the circuit with reserve, it is rejected and the diagnosis is kept.

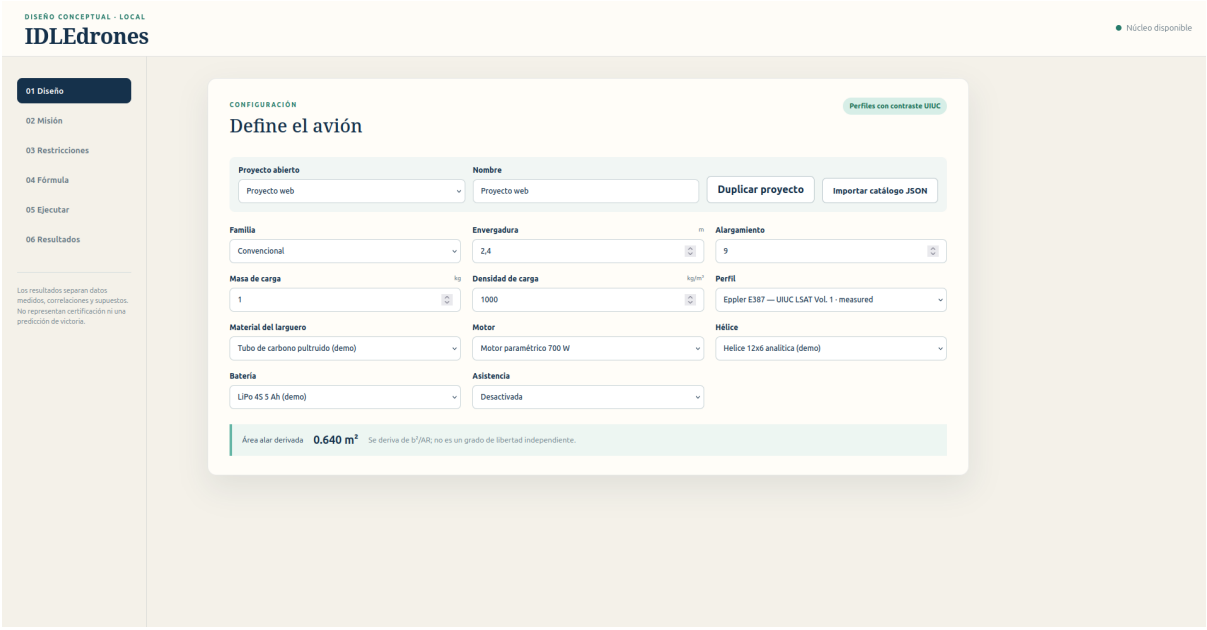


Figure 1: IDLEDrones design editor

The editor defines the family, the geometry, the payload and the main components of the aircraft. The interface is in Spanish. Component names such as "LiPo 4S 5 Ah (demo)" or "Motor paramétrico 700 W" are generic labels the AI gave to the catalogue entries, not commercial products.

The optimiser combines continuous variables with catalogue components, compares the families separately and refines the best candidates with higher-fidelity models. From the web you can edit the scoring formula and the constraints, launch evaluations or optimisations, simulate wind with Monte Carlo, cancel and resume jobs, compare results and export them as JSON and PDF. Every run keeps its configuration, seeds, catalogues, model versions and hashes so it can be reproduced and audited.

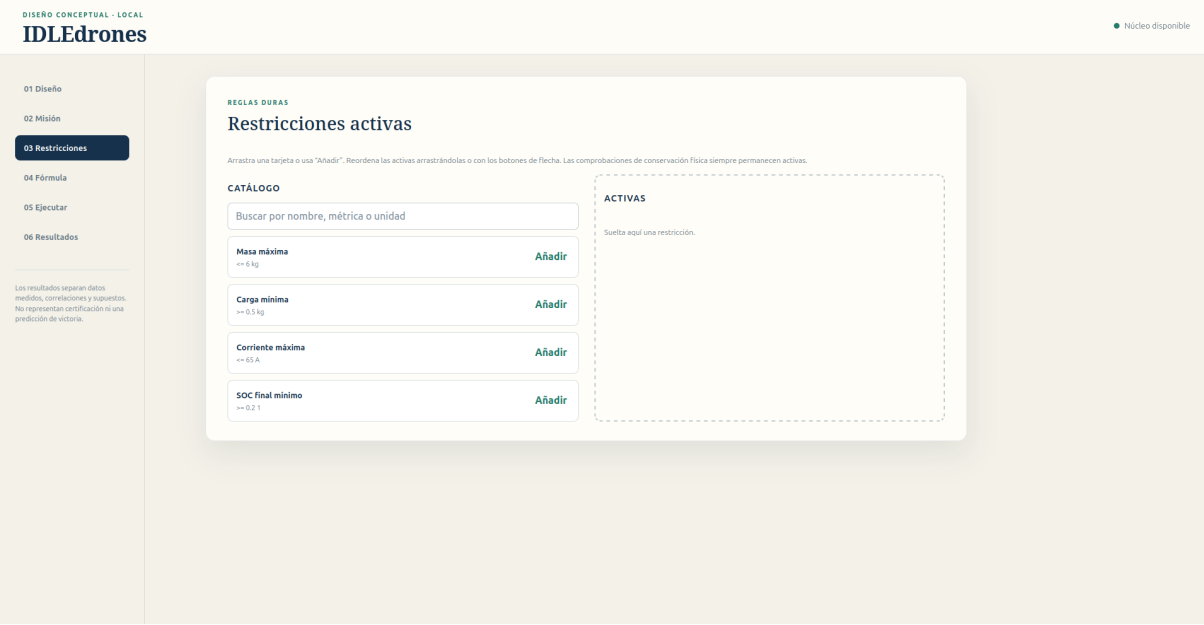


Figure 2: IDLEDrones constraint editor

Constraints are added from a catalogue by dragging cards; the basic physical checks always stay active.



Figure 3: IDLEDrones evaluation result

The application shows the metrics of the evaluated candidate, its margins and the selected components. Identifiers like `motor-1000w-demo` *or* `lipo-4s-3ah-demo` *are, again, generic names the AI assigned.*

Analysis

One of the most fascinating things is that it incorporated real, publicly available data from manufacturers and experimental repositories, as well as physical equations and correlations. It is not just a toy: it is an engineering tool at an early stage. Even so, the current comparisons are mostly per component, and they do not yet validate the accuracy of the aircraft as a whole.

The drones it proposes make sense from an engineering point of view, but for now it is wise to be sceptical, because the tool shows the winner and little else. Without evidence that it really is the best, "it is the optimum" remains a questionable claim.

Another limitation of the verification is that, without me knowing it during the run, the environment had no access to a graphical browser. The web flow was checked through API tests and tests of the built application, but not through a real interactive session in a browser. The visual review was limited to the rendered PDFs.

At a midpoint I counted 34 tests. The delivery ended with 47 passing tests, which is still modest coverage for a tool you would want to consider critical. If I started again, I would push earlier for building the models with more engineering rigour. I would probably go from **documentation → tool** to:

1. Documentation.
2. Scientific cross-checking of formulas and data.
3. Incremental implementation with tests.
4. Integrated validation.

I would probably keep GPT-6 Astra for the documentation and use GPT-5.6 Sol for the other three phases.

Something else to improve is that there are still some heuristic constants inside the models, such as the size of the speed grid or the discretisation of the turns. Not everything is **hardcoded**: catalogues, components, mission parameters, constraints and the main physical options are editable. Still, it would be worth identifying and centralising the constants that affect numerical fidelity, so they can be reviewed and studied easily.

As a starting point the result is impressive, but there is still work to do for it to be really useful. The weakest part is the interface: it mixes editing a specific aircraft with defining the space of an optimisation. For example, asking upfront for the wingspan and the dimensions suggests you have to fix the aircraft before optimising it, when they should be ranges, limits or variables the optimiser decides. It also lets you pick a base family while the optimisation can explore all three, without explaining the relationship well.

The foundation is there, but there is a lot of work left

The tool works, but it leaves several clear directions for improvement:

- **Aerodynamics and structures.** It uses reasonable conceptual models to explore designs, but it lacks more detailed aerofoils and aerodynamic effects, as well as manufacturable structures with joints, adhesives and local details.
- **Validation.** It would need more bench and flight data to check the complete aircraft, not just its models separately.

- **Code and tests.** The architecture is already split into modules, but its interfaces should be hardened and the regression and integration tests extended.
- **Optimisation and computation.** If much larger spaces had to be explored, distributed computing, vectorisation or GPUs could be studied; adding a GPU is not enough if the models are not adapted to it. The optimiser should also justify its result, with a comparison against the discarded candidates, the sensitivity to the score weights and whether the optimisation has converged.
- **Interface.** It needs a clear separation between "analyse this aircraft" and "find the optimal aircraft".

These are not features to add for the sake of it, but possible paths to turn this first conceptual version into a more reliable tool.

All in all, the tool has saved an enormous amount of work, but right now it is nowhere near what it aimed to be. The interface and the physics still need improving and, above all, part of what was built needs to be steered towards something more serious and reliable.

Conclusions and advice

Don't be afraid to ask

AI is very powerful, more than we want to believe. Don't be afraid to ask for something in case the AI isn't up to it. Worry, above all, about documenting well how you want it to work.

Nor should you be afraid to ask because of the vertigo that the AI might do it better than you. The sooner you accept that it can do in seconds things that would take you days or weeks, the better.

Be precise and rigorous as early as possible

However much context the models can handle, the best time to make changes is at the beginning. Spending time on documentation and planning is worth it. That is when the project is at its most compact; later there will be a huge amount of implementation noise.

You have to find a middle ground between this advice and the previous one. The more you know about the subject, the more precise you should be. The fewer details you give, the more freedom the model has, and that can turn out well or badly. In my view, it pays to be precise about what you know and to leave it free to give its opinion on what you don't.

Don't build for the sake of building

Having such a powerful tool creates the temptation to build ever bigger things. It has happened to me that, before I realised it, I had added unnecessary features that only bloated the result and turned the tool into an unbearable brick. Don't lose sight of what you are trying to do. Design a structure that can grow and lets you add things in the future, but start with something small. Think big, act small.

Reflection

It is essential to keep studying and learning new things, because however far AI advances, you can never ask for something you don't know exists. It sounds silly, but it matters. At least for now, we are the ones who decide what to do: we can delegate the **how** to the AI, but we still decide the **what**. You cannot want a fixed-wing drone if you don't know fixed-wing drones exist. If you only say "drone", the AI may lean towards a quadcopter.

On the other hand, the space of possibilities is practically infinite. A problem can have countless solutions and ways to approach it. As you add constraints, concrete objectives and context, you close that space. Asking an AI for the optimal solution to your problem in five lines will almost never work, because it lacks an enormous amount of context that you do have. Being precise when framing the problem and learning about the subject will remain useful. It is not only a question of the AI's capability, but of the space of possibilities and the information available. Models will have more and more intuition, but they cannot faithfully rebuild a context you never gave them.